

Demo video — final consolidated script and production plan (rev 4)

Target runtime **3:55**. RULES VERIFIED (Devpost page, 2026-08-20): the requirement is “~ **4-min Demo video**” (approximate — no hard cutoff stated) containing the problem overview, the value proposition, the app in action, and — mandatory — a demonstration that “the backend is running on Google Cloud (ie: Google Cloud Console, Cloud Run dashboard, Vertex AI logs, URL of .run, etc)”. This script satisfies that three ways: the `.run.app` address bar throughout the take, the Cloud Logging cutaway, and the Cloud Run console shot. No hosting/visibility rule is stated for the demo video itself (the public-not-unlisted rule applies to the BONUS content piece); public upload stays the safe default. The judging rubric asks verbatim for “a live, unedited demo ... and visible proof it runs on Google Cloud” (Demo & Production Readiness, 30%). Deadline: Aug 31, 2026 @ 5:00pm PDT.

Audience: judges with zero prior context. The one thing this video must do that nothing else can: **prove the real Gmail → Google Cloud → Calendar path executes**, in a continuous unedited span, with Google Cloud visible.

Editorial rule for every beat: **show the engineering; narrate the consequence**. The four memories a judge must leave with: it reads real commitments out of email · it refuses to guess (requests ≠ commitments, elapsed ≠ done) · it puts real work time on a real calendar · when the calendar changes, it fixes the plan itself in seconds. Everything else is a credibility enhancer, not the story.

Timeline

Time	Beat	Surface
0:00–0:22	Hook + what it is (≤58 words, ~160 wpm)	You / title card
0:22–2:47	THE CONTINUOUS TAKE (order below; final-take budget ≤2:25)	Gmail, dashboard, Calendar
2:47–2:54	Cloud Logging cutaway (three highlighted rows)	Cloud Logging
2:54–3:19	Sandbox: live Gemini on a novel message (precision claim as caption, not speech)	/sandbox (staged minutes before)
3:19–3:45	Five-node architecture + on-screen stat card	Diagram + Cloud Run console
3:45–3:55	Closing line + links	Title card

Inside the continuous take, in this order (the order is load-bearing):

1. Send the request email (counterparty account, clean compose popup)
2. Candidate appears on dashboard with its evidence quote
3. Accept by reply → same commitment updates, no duplicate
4. Confirm effort (type minutes) → planner runs → approve first plan
5. Google Calendar: the real blocks
6. **Completion beat on the pre-staged commitment** — introduced with the word “Separately”; give it a visually distinct title (e.g. new commitment “Q4 budget summary” vs staged “Finalize onboarding brief”)
7. Quiet window (~20s): the audit-trust beat — NO calendar writes here; this protects the next step from push throttling
8. **Drop a meeting onto a reserved block** → **autonomous repair** (~9s)

9. Expand the plan_repaired audit entry: planner run, moved/preserved blocks, stable event ids, and the system's own measured repair_latency_ms

Script

0:00–0:22 — Hook + what it is

(You on camera or VO over your inbox, then title card. ≤58 words at a calm ~160 wpm — do not inflate; the demo carries the rest.)

“I promise things over email all the time — ‘sure, I’ll get you the deck by Friday’ — and then it just sits in my inbox. My calendar has no idea the work exists.

CommitmentOS finds those promises, books work time on my Calendar, and repairs the plan itself when things change. First: the deployed system, running continuously against my real Gmail and Calendar.”

(This wording survives the split-take fallback — the “no cuts” claim lives only in the caption over the actual continuous frames.)

0:22–2:47 — THE CONTINUOUS TAKE

(One screen recording. Caption overlay spanning exactly the continuous frames: “one continuous take — real accounts, deployed service”. Address bar visible. 110–125% browser zoom. Final-take budget ≤2:25.)

1. Send the request (counterparty Gmail, clean compose popup — never show the inbox list) > “A colleague asks me for something.” (type and send: “Hi — could you put > together the Q4 budget summary? We need it by [day].” — pick the day per > the date-math note) “Watch the deadline — ‘by [day]’ — just human > language.”

2. Candidate appears (dashboard Today tab; expect 10–30s: push + interpretation + 8s poll. If the wait runs long, use the filler line — never silence over a static screen.) > “That message is being pushed to the agent right now...” (filler if > needed: “...Gmail sends a change signal to the service on Cloud Run; the > agent fetches just that thread and reads it...”) “...there. And notice: it > is NOT my commitment yet — it’s a request, held as a candidate.” > (CLICK INTO the candidate — its detail page shows the Source evidence > card) “...with the exact sentence it’s based on. The agent won’t invent > obligations for me.”

3. Accept by reply (my Gmail: “Yes, I’ll take that on — you’ll have it by [day] end of day.” Back to the commitment DETAIL page — the evidence stays visible and the approval forms live here too.) > “My reply updates the **same** commitment instead of creating another one > — one promise, one record, now mine.”

4. Effort + plan approval (on the detail page; effort field is blank — that’s the point; type the minutes; brief planner wait before the plan card. VERIFY IN REHEARSAL that both approvals surface on the detail page; fall back to Today’s approval cards if not.) > “It won’t guess how long my work takes — I tell it: three hours. ...It’s > computing a plan against my real calendar right now... and here’s its last > **setup** question: may I write to your calendar? After this, in-policy > repairs happen on their own — and anything outside policy still comes > back to me.” (approve)

5. Real Calendar (Calendar tab) > “The blocks land here — real events, before the deadline, around > everything already on my calendar.”

6. Completion beat (dashboard → the PRE-STAGED commitment; distinct title so the switch is visually unmistakable; overlay caption: “Separate commitment — completion integrity”) > “**Separately** — here’s an older commitment, showing another control. > One of its blocks elapsed this morning, and the agent > did NOT assume the work happened. It’s asking me. I log the 60 minutes I > actually did... remaining work drops by exactly 60. And when I mark it done > — it keeps my real number, and cancels the reserved time I no longer > need.” (Calendar tab: the future block is gone) “Time passing never > counts as progress. Only I do.”

7. Quiet window — the trust beat *(expand the completion audit entry; ~20s, no calendar writes; point the cursor along the chain as you speak)* > “Everything it does lands on an audit timeline. These entries connect the > input it observed... the decision it made... the calendar action it wrote... > and the verified result — under one correlation id. You never have to > take the agent’s word for what happened.”

8. THE CLIMAX — conflict → repair *(Calendar tab: drag a new meeting directly onto a reserved block; hands off; count through it)* > “Now the part I built this for. I’ll drop a meeting right on top of the > time it reserved — the kind of thing a coworker does to your week. Hands > off now... *(wait; ~9s warmed)* ...there. The agent noticed through the > calendar’s own change feed, moved exactly that one block, and left the > others untouched. I never clicked replan — the calendar change itself > triggered the repair.”

9. The receipt *(dashboard Activity: expand plan_repaired; read the moved/preserved counts and the latency FROM THE SCREEN, never from memory)* > “And it can prove it: the planner run, the one moved block, the preserved > ones, the stable event identity — and the repair latency the system > measured on itself: [read the real number] seconds.”

(If the pause switch comes up at all, it is narration ONLY — do not toggle it: “And if I ever want it to stop — one switch pauses automatic action; anything in flight is held, not lost, and revalidated before it resumes.”)

2:47–2:54 — Cloud Logging cutaway

(Separate ~7s clip. A visual receipt, not an explanation: the smallest credible view — three rows highlighted, timestamps aligned: webhook in → reconciliation → calendar write. PRE-SCRUB the visible log lines for email addresses, ids, tokens, or query parameters before recording.) > “Same story from Google Cloud’s side — the webhook, the reconciliation, > the calendar write, seconds apart.”

2:54–3:19 — Sandbox: prove the AI is live

(Separate take. Stage the session MINUTES before recording — sandbox worlds idle-expire and reads don’t refresh the clock; a session staged in the morning will be dead by afternoon. Free-play lane chosen, subject entered. On camera: type only the message, then expand the interpretation evidence. The precision claim rides as an on-screen caption, not speech: “Same model · same prompt contract · isolated test data · no login”.)

“To show the interpretation happening live, I’ll give the public sandbox a message it has never seen...” *(type as counterparty: “Could you send me the revised onboarding guide by next Wednesday at 3pm?”)* “...interpreted live by Gemini: there’s the model, the wall-clock latency, and the exact words it cited.”

(If the label shows recorded-fallback : transient provider hiccup — wait ten minutes or reset, then re-record this segment.)

3:19–3:45 — Architecture + the stat card

(Five-node diagram `architecture_five_node.png`; the implementation nouns — idempotent tasks · transactional outbox · etag preconditions — appear as a caption strip, not speech. Then 2s of the Cloud Run console showing the service + revision. STAT CARD on screen, scoped exactly:)

On-screen card: 10 live end-to-end runs + 10 hardened seeded runs · 1,110 acceptance checks · 0 duplicate commitments or events · 73/73 security probes · ~\$0.0008 per message

“One loop: Observe, Interpret, Plan, Policy, Execute — and every result is observed again, so the loop closes. The rule that matters: Gemini only ever interprets language. Everything that touches the calendar is deterministic code, and etag preconditions reject stale writes — an older plan can’t overwrite a newer calendar change. The receipts are on screen, and every number is in the repo, indexed to the exact deployment that produced it.”

“CommitmentOS doesn’t plan once. It keeps the promise feasible as reality changes.”

Pre-production (day before)

1. **Clean up, THEN stage** — run the audited controlled-account cleanup first; then create the pre-staged commitment (~120 min effort: one block that elapses the morning of recording + one later-in-week block that will visibly vanish on completion). Give it a title visually distinct from the new commitment’s. Staging after cleanup, never before.
2. **Warm the instance:** `--min-instances 1` (verify actual current state; revert after recording).
3. **Verify:** monitoring ACTIVE and actions enabled (a paused monitor once silently killed a whole campaign); Gmail/Calendar watches current (renewal job is daily); zero stale pending approvals on Today; Calendar UI in Pacific.
4. **Date math:** choose the deadline expression for your recording day so it lands days out (“by Friday” from a Monday–Wednesday; otherwise “by next [day]”). More days = more visible placement spread.
5. **Accounts:** sign into `/app` fresh (12h session TTL). Counterparty sends from a separate browser profile — do NOT touch its OAuth (a wrong-account reconnect once poisoned a full night of runs). Set friendly display names on both accounts; the real controlled address WILL be on camera — deliberate choice, not an accident.
6. **Tabs prepped:** counterparty compose popup · dashboard · Calendar · Cloud Logging with the filter query already typed · Cloud Run console. Clean profile, notifications off, canonical host in the address bar (`commitmentos-2hscowvydq-uw.a.run.app`).
7. **Rehearse the full take twice, recorded at full quality** — a rehearsal where everything lands IS a usable final take. Keep every take.

Rehearsal pass criteria (two tiers)

- **Engineering pass** (the system works): push-to-candidate < 30 s; conflict-to-repair < 15 s; take completes < 2:45.
- **Final-take pass** (the footage fits the edit): all of the above AND the continuous take ≤ **2:25** (2:20 preferred, for edit tolerance). A take that passes engineering but runs long is a rehearsal, not a keeper.
- **Repair slower than ~15 s = failed FINAL take.** During rehearsal, keep rolling anyway — the once-a-minute safety reconciliation catches a suppressed push within ~60–75 s, and watching it happen is how you diagnose the delay. Footage of a 60 s repair is the emergency last resort only if no take ever passes.
- Two engineering failures → the **split fallback**: two continuous takes with the seam after beat 5 (Calendar view), each captioned honestly (“continuous — no cuts”) — never cut inside beats 8–9. The intro makes no one-take claim at all, so the split needs no intro re-record.
- Verify during rehearsal: completing the staged commitment does NOT shift the new commitment’s blocks (preserve-then-allocate should hold them).

Overlay & asset inventory (produce BEFORE recording day)

1. Opening title card (name + one-line tagline)
2. Continuous-take caption: “one continuous take — real accounts, deployed service”
3. Completion overlay: “Separate commitment — completion integrity”
4. Sandbox caption: “Same model · same prompt contract · isolated test data · no login”
5. Implementation-nouns strip (architecture beat): “Idempotent tasks · Transactional outbox · Etag preconditions”
6. Stat card: “10 live end-to-end runs + 10 hardened seeded runs · 1,110 acceptance checks · 0 duplicate commitments or events · 73/73 security probes · ~\$0.0008 per message”

7. Closing card: tagline + sandbox link, demo link, repo link (canonical host)

Recording notes

- 1080p minimum, browser at 110–125% zoom (embedded players compress), cursor visible.
- Live mic during the continuous take (authenticity through the waits); clean VO for intro/architecture/close is fine.
- No copyrighted music.
- Captions/subtitles recommended.

Post-recording

- Confirm monitoring active + actions enabled (mandatory if anything was toggled).
- Revert `--min-instances` if desired.
- Upload **public** on YouTube (confirm the rules, but public is the safe default for judging — an unlisted link risks a rule technicality), test playback on a cold browser, then put the link in Devpost and the LinkedIn post.

Truthfulness guardrails (non-negotiable)

- The “no cuts” caption spans exactly the frames that are continuous; the spoken intro makes no one-take claim, so it stays true under the fallback.
- Narrate what the viewer sees: YOU drop the conflicting meeting (don’t say “someone books a meeting” while visibly dragging it yourself). A counterparty calendar invite is an UNTESTED conflict path — do not attempt it for the video.
- The repair-trigger line is “I never clicked replan — the calendar change itself triggered the repair” (states the mechanism); never “nobody opened an app” or “nobody opened CommitmentOS” — the interface is visibly open.
- Mechanisms, not absolutes: “etag preconditions reject stale writes,” not “can never overwrite.”
- The stat card scopes the campaigns explicitly: 10 live end-to-end + 10 hardened seeded. Scoped evidence reads stronger, not weaker.
- Never call `/demo` “live” — it is seeded; the sandbox and the continuous take are the live proof.
- Read the repair latency number that actually appears on screen.
- “Same model, same prompt contract” is the exact sandbox claim (production interpreter class and prompt on a sandbox-only key) — not “same system.”
- **One description per surface, used consistently everywhere:**

Surface	Correct description
Real-account demo	Production service connected to real Gmail and Calendar
<code>/sandbox</code>	Public interactive environment — same model and prompt contract, isolated test data
<code>/demo</code>	Seeded, read-only product walkthrough

- The stat card carries the numbers; don’t re-speak them — spoken narration stays at the two-ideas-plus-pointer level.